

ReadMe File – A description of my C coursework

Compiling

My program contains 4 source files: graphics.c, Visuals.c, Maze.c and Main.c. Therefore to compile my program you must type **gcc graphics.c maze.c visuals.c main.c**. To run the compiled program you do not have to type in a start position for the robot as there is an initial position, however if you wish to type in a position for the robot to start **./a.out 5 4 | java -jar drawapp-2.0.jar**, with the 5 representing the x coordinate and the y representing the y coordinate for example would work. My grid coordinates start at 0 on both the x and y axes. I had to edit the graphics.h file with **#ifndef**, **#define** and **#endif** so that it wasn't included more than once when compiling.

Changing Values

In my Maze.h file I have defined **X_SIZE**, **Y_SIZE**, **X_HOME**, **Y_HOME** as well as the number of **BLOCKS**. All of these values can be successfully changed and allow the program still to work. However, there is a maximum window size so if you were to make the grid very big, you may need to also change **BLOCK_WIDTH** in Visuals.h. If you decrease block width then my blocks will also decrease. However, you would need to decrease the size of my images of home and the robot to then account for this. My home and robot are set to be **(BLOCK_WIDTH-10 x BLOCK_WIDTH -10)** in terms of their pixel dimensions. Furthermore, if you have too many blocks my robot may not be able to find a path home, depending on the initial position of the robot and the position of home, this usually takes around 15-20 blocks. The grid needs to be a square because of the way I have done indexing.

Explaining my Algorithm

Most of my logic is in my Maze.c file. The **FindShortestPathHome** and **VisitUnblockedSquares** are my most important functions. Starting with the immediate neighbours of the home square, I set the move vectors for each of these squares. If the robot finds itself in the North, East, South, or West neighbours, a simple one step down, up, left, or right respectively will get it to the home square. Therefore the move vectors for these neighbours are respectively (0,1), (1,0), (0,-1), (-1,0). If the robot finds itself in the corner neighbours of the home square, since diagonal moves are not permitted, it must first move clockwise, to the East, South, West, or North neighbours, and then from there move to the home square. So, it identifies a neighbouring square that has been visited and moves towards it. I repeat above steps for each of the neighbours of the neighbours of the home square and so on. At the end, relative to the location of the home square and blocked squares, each unblocked square will have a move vector that when followed guarantees the robot will reach home in the shortest possible steps from that square. The function needed to control my robot is simple **MoveRobot** in my Visuals.c file. This function moves the robot one step at a time in the correct direction, in a way it combines together **void forward()** **void left()** and **void right()**.

```
void FindShortestPathHome(Square *pSquares) //main code for my algorithm
{
    int unblocked = CountUnblocked(pSquares);
    const int maxLayer = CountLayers(pSquares);
    int layer = 0;
    while(layer < maxLayer)
    {
        layer ++;
        for (int k=0; k<4; ++k)
        {
            for(int n=0; n<layer*2-1; ++n )
            {
                int index = FindNeighbourIndex(pSquares, layer, k, n);
                if (AvailableIndexOrBlock(pSquares, index) == false)
                {
                    continue;
                }
                if (layer == 1)
                {
                    unblocked = LayerOneMoveVectors(pSquares, unblocked, index, k);
                }
                else
                {
                    unblocked = VisitUnblockedSquares(pSquares, index, unblocked);
                }
            }
        }
        for (int k=0; k<4; ++k)
        {
            unblocked = CornerMoveVectors(pSquares, unblocked, layer, k);
        }
    }
    while(unblocked > 0)
    {
        for(int i=0; i<X_SIZE*Y_SIZE-1; ++i)
        {
            unblocked = VisitUnblockedSquares(pSquares, i, unblocked);
        }
    }
}
```